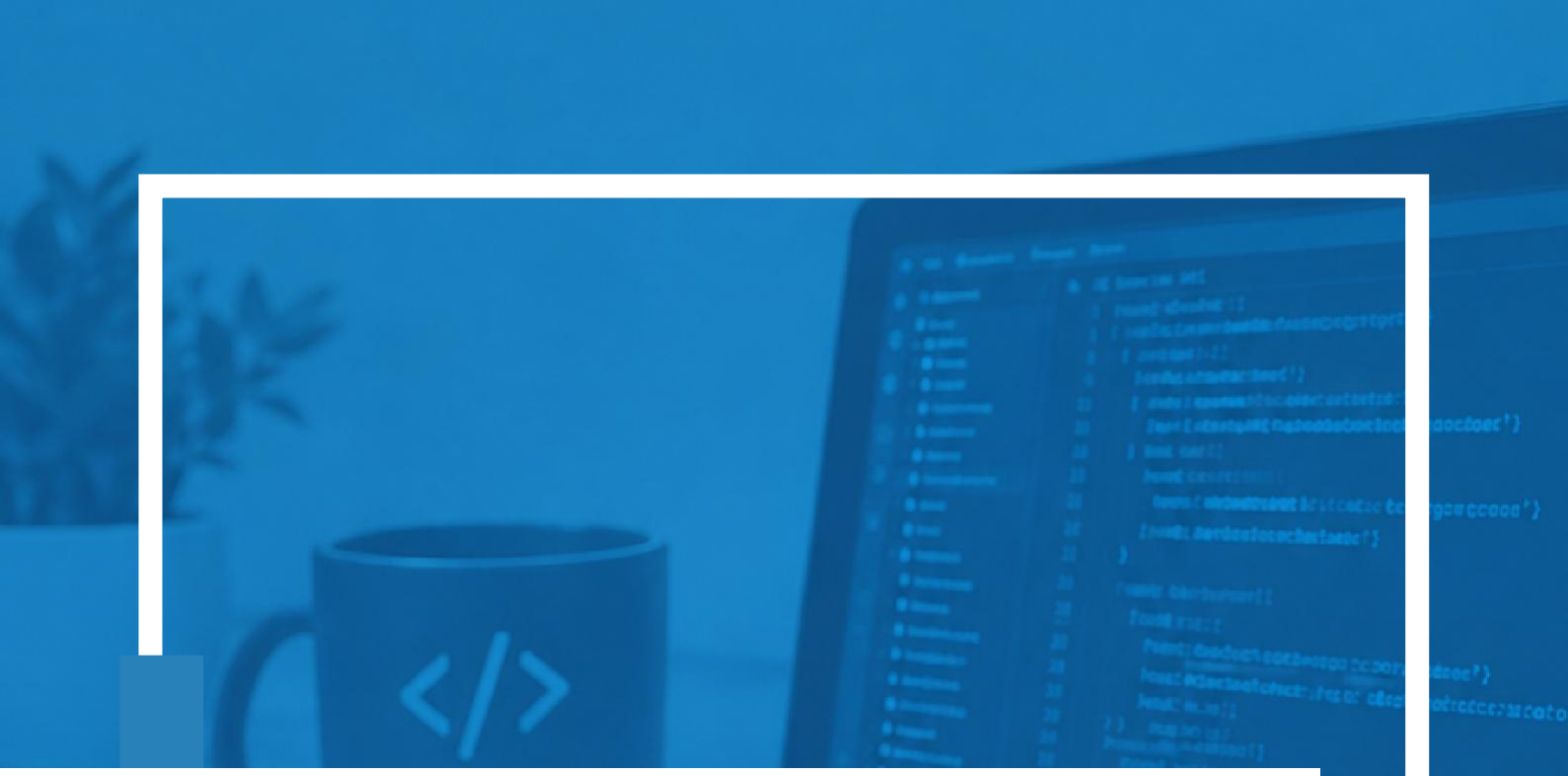


Fase 3

Repita Comigo: Laços de Repetição

Try, Catch & Learn

Fase 3
Repita Comigo: Laços de Repetição

Automatizando tarefas repetitivas com laços controlados por contador e por condição, sem perder o controle do fluxo.

1. Quando precisamos repetir?

Até agora, nossos programas conseguem:

- receber informações;
- realizar cálculos;
- tomar decisões com **if** e **else**.

Mas imagine que precisamos mostrar uma mensagem **10 vezes**.

Sem um laço, teríamos que escrever:

```
printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");

printf("Olá!\n");
```

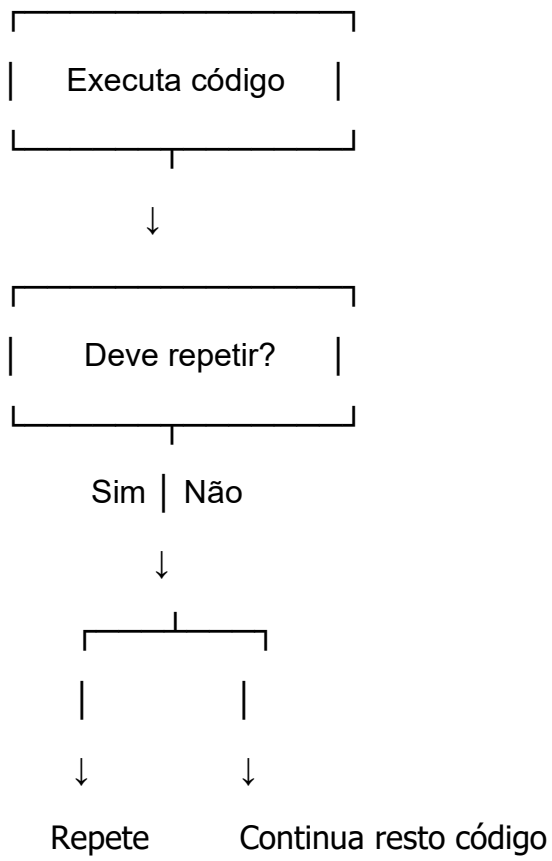
Isso funciona, mas não é uma forma prática de programar.

Os **laços de repetição** permitem executar um mesmo trecho de código várias vezes de maneira automática.

2. O que é um laço de repetição?

Um **laço de repetição** é uma estrutura que permite que um conjunto de instruções seja executado várias vezes.

A ideia básica é:



O ponto mais importante é:

Um laço precisa ter uma forma de controlar quando deve continuar e quando deve parar.

3. `while` — enquanto uma condição for verdadeira

O `while` significa **"enquanto"**.

Ele repete um trecho de código **enquanto uma condição for verdadeira**.

```
while (condicao) {
    // código que será repetido
}
```

Por exemplo:

```
int contador = 1;
while (contador <= 5) {
   printf("%d", contador);
   contador++;
}
```

Resultado:

1 2 3 4 5

Mas o que aconteceu nesse laço?

contador = 1

↓

1 <= 5? → SIM → mostra 1

↓

contador = 2

↓

2 <= 5? → SIM → mostra 2

↓

contador = 3

↓

3 <= 5? → SIM → mostra 3

↓

...

↓

contador = 6

↓

6 <= 5? → NÃO

↓

PARA

Mas cuidado! Todo laço precisa atualizar o contador: **"contador++;"**.

Caso contrário, a condição pode continuar verdadeira para sempre, criando um **laço infinito**.

4. Contador

Um contador é uma variável utilizada para acompanhar quantas vezes algo aconteceu.
int contador = 1;
Podemos aumentar seu valor usando: **contador++;**
Isso equivale a: **contador = contador + 1;**
Também podemos diminuir: **contador--;**
Isso equivale a: contador = **contador - 1;**

5. while controlado por condição

Nem sempre sabemos exatamente quantas vezes o código será repetido. Às vezes queremos repetir até que alguma condição aconteça.

```
int numero = 1;
while(numero != 0){
        printf("Digite um numero: ");
        scanf("%d", &numero); }
}
```

Nesse caso, o programa continua pedindo números enquanto o usuário **não digitar 0**.

```
Digite um numero: 5
Digite um numero: 8
Digite um numero: 3
Digite um numero: 7
Digite um numero: 0

Programa encerrado.
```

Aqui, o número de repetições **não foi definido antecipadamente**.

6. do while — execute pelo menos uma vez

O **do while** é parecido com o while, mas existe uma diferença importante: no do while, o **código é executado antes de a condição ser verificada.**

```
do {
        //código que será executado
} while (condição);
```

Exemplo:

```
int numero;

do {

        printf("Digite um numero positivo: ")
        scanf("%d", &numero);

}while (numero <= 0);
```

Nesse exemplo, o programa sempre fará a pergunta pelo menos uma vez.

Diferença entre while e do while

while:
Verifica → Executa → Verifica → Executa …

do while:
Executa → Verifica → Executa → Verifica …

7. for — repetição controlada por contador

Quando sabemos **quantas vezes queremos repetir algo**, o **for** costuma ser uma opção prática.

```
for (inicialização; condição; atualização){
        // código
}
```

Exemplo:

```
for (int contador = 1; contador <= 5; contador++){
        printf("%d", contador);
}
```

Resultado: 1 2 3 4 5

O for reúne três partes:

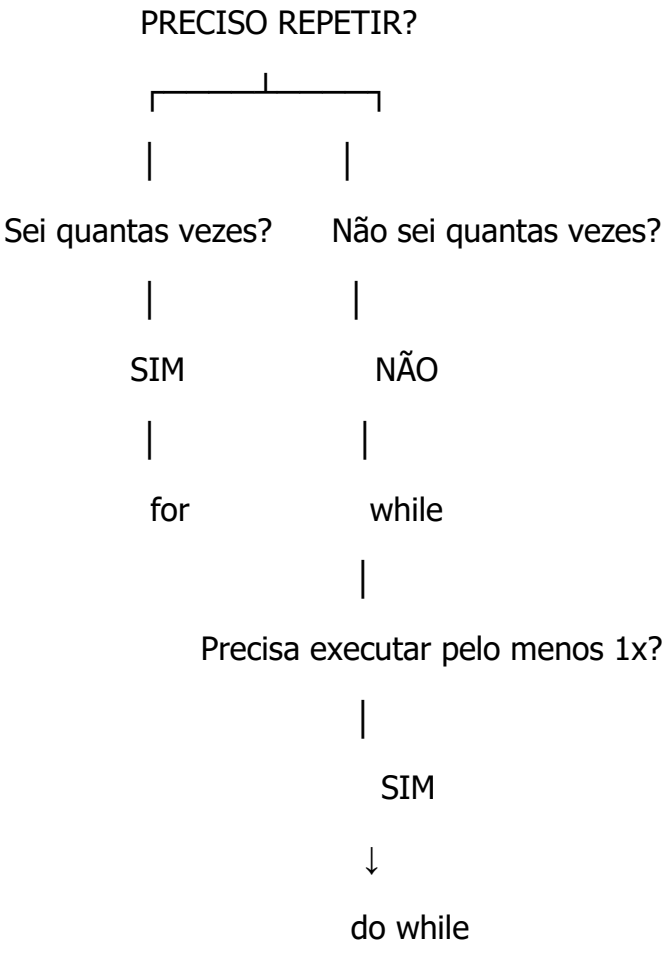
for (inicialização; condição; atualização)

↓ começa ↓ verifica ↓ atualiza

8. Comparando os três laços

Antes de ensinar o programa a decidir, vamos ensiná-lo a calcular. Os operadores básicos são: + soma, - subtração, * multiplicação, / divisão, % resto da divisão

Laço	**Quando usar?**
while	Quando a repetição depende de uma condição.
do while	Quando o código precisa executar pelo menos uma vez.
for	Quando a repetição é controlada por um contador



9. break — pare o laço

O **break** interrompe imediatamente o laço.

```
int numero;

while (1){

        printf("Digite 0 para sair: ");
        scanf("%d", &numero);

        if (numero == 0){
                break;
        }
}
```

Quando o usuário digita 0, o break encerra o laço.
break significa: pare a repetição agora.

10. Primeiro exercício de lógica

O **continue** faz o programa pular o restante daquela repetição e voltar para a próxima.

```
for (int i = 1; i <= 5; i++){

        if (i == 3) {

                continue;

        }

        printf("%d", i);

}
```

Resultado:

1 2 3 4 5

O número 3 não foi mostrado porque o **continue** fez o programa pular aquela repetição.

continue significa: ignore o restante desta repetição e continue para a próxima.

11. Pensando como um programador

Antes de criar um laço, faça três perguntas:

1. **O que precisa ser repetido?**

   Mostrar um número na tela.

2. **Quando deve continuar?**

   Enquanto o contador for menor ou igual a 10.

3. **Quando deve parar?**

   Quando o contador passar de 10.

Depois transformamos isso em código:

```
for (int contador = 1; contador <= 10; contador++){
        printf("%d", contador);
}
```

12. Exercícios

Exercício 1 — Contagem

Faça um programa que mostre os números de **1 até 10** usando **for**.

Exercício 2 — Contagem regressiva

Faça um programa que mostre: 10 9 8 7 6 5 4 3 2 1

Exercício 3 — Soma

Faça um programa que leia **5 números** e mostre a soma deles.

Digite um numero: 2

Digite um numero: 5

Digite um numero: 3

Digite um numero: 4

Digite um numero: 6

Soma = 20

Exercício 4 — Senha

Faça um programa que peça uma senha ao usuário. Enquanto a senha estiver incorreta, continue pedindo. Quando a senha estiver correta, mostre: **Acesso permitido!**

Exercício 5 — Número positivo

Faça um programa que peça um número ao usuário. Enquanto o número for menor ou igual a **0**, peça novamente.

Quando o usuário digitar um número positivo, mostre: Número válido!

O que você já consegue fazer:

Ao terminar esta fase, você já consegue:

- entender o conceito de repetição;
- utilizar **while;**
- utilizar **do while;**
- utilizar **for;**
- trabalhar com contadores;
- controlar quando um laço começa e termina;
- utilizar **break;**
- utilizar **continue;**
- criar programas que repetem tarefas automaticamente

Até aqui, o programa já sabe existir, calcular, tomar decisões e repetir tarefas.

Na próxima fase, vamos começar a organizar e trabalhar com vários dados de uma forma mais estruturada.